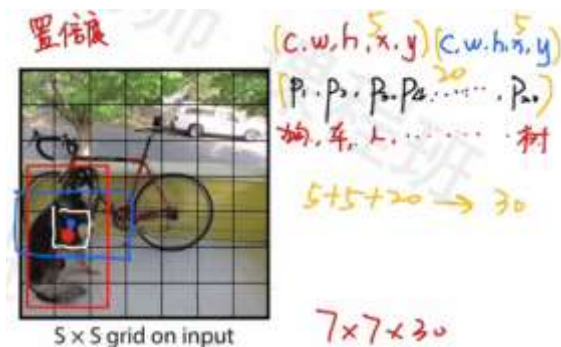
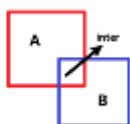


01. 根据视频（048YOLO模型输出的具体含义）中所讲述的内容，回答：模型的输出与原图的关系是怎样的？



答：每个格子有 30 个通道，通道 1-5 表示红框画法的时候输出的 (C,W,H,X,Y), 通道 6-10 表示蓝框画法的时候输出的 (C,W,H,X,Y)，剩余的通道 11-30 表示 20 个物体分类的类别概率。

02. 根据视频（050 IOU交并比）定义计算两个box的IOU值的函数。



红色框 的面积为 S_A

蓝色框 的面积为 S_B

交集部分（两个框重叠的矩形区域）的面积为 S_{inter}

$$IOU = \frac{S_A \cap S_B}{S_A \cup S_B} = \frac{S_{inter}}{S_A + S_B - S_{inter}}$$

03. 如果当前数据中拥有多个类别，NMS过程要如何进行？

一般是拆解成多个单类别的 NMS 来解决，对应着两个循环，外面的 for 循环是遍历每个类别（狗、猫、房子等），内部的 while 循环是针对类别内部进行循环 NMS 计算

- 对 Group A 内部执行一次 NMS，剔除多余（重叠度高但置信度低）的狗。
- 对 Group B 内部执行一次 NMS，剔除多余（重叠度高但置信度低）的人。
- 对 Group C 内部执行一次 NMS，剔除多余（重叠度高但置信度低）的房子。

04.YOLOv1的模型训练过程与预测过程有什么区别？

首先是目的不同

预测过程：通过输入 `img` 进入模型后，找出最终的目标检测结果 (`C,W,H,X,Y`)

训练过程：观察实际输出与理想输出的差异，找出代价函数，并通过反向传播降低损失来更新模型参数

再者是对于边界框的处理不一样：预测过程会通过 NMS 去掉重叠高置信度低的框，而训练过程对于所有框都要保留来计算代价函数。

05.YOLOv1的代价函数中每一项的作用是什么？请分别解释。

1. 坐标误差 (Coordinate Error)

这一项用于衡量模型预测的边界框位置和大小与真实标签之间的差距。

2. 置信度误差 (IOU Error / Confidence Error)

这一项用于衡量模型对“框内是否有物体”以及“物体位置准确性”的预测信心。它被拆分为两部分计算：

- 包含目标的置信度误差

作用：当网格中确实存在目标时，计算预测置信度 (`C`) 与真实置信度（期望为 1 或 IOU 值）之间的误差。

目的：鼓励模型在有物体的地方输出高置信度。

- 不包含目标的置信度误差 (Background Error)

作用：当网格中不包含目标（即背景）时，计算预测置信度与 0 之间的误差。权重 λ_{noobj} ，由于图片中大部分区域都是背景（不包含目标），如果不加控制，背景的负样本会主导梯度，导致模型难以训练。因此，引入了权重参数 λ_{noobj} （通常为 0.5），降低背景误差对总损失的贡献，以解决正负样本不平衡的问题。

3. 分类误差 (Classes Error)

这一项用于衡量模型对目标类别的识别准确性。计算预测的类别概率（如猫、狗、人等 20 个类别的概率）与真实类别标签之间的均方误差

06. 通过所学知识分析，为什么YOLOv1对密集场景中的小物体检测较差？

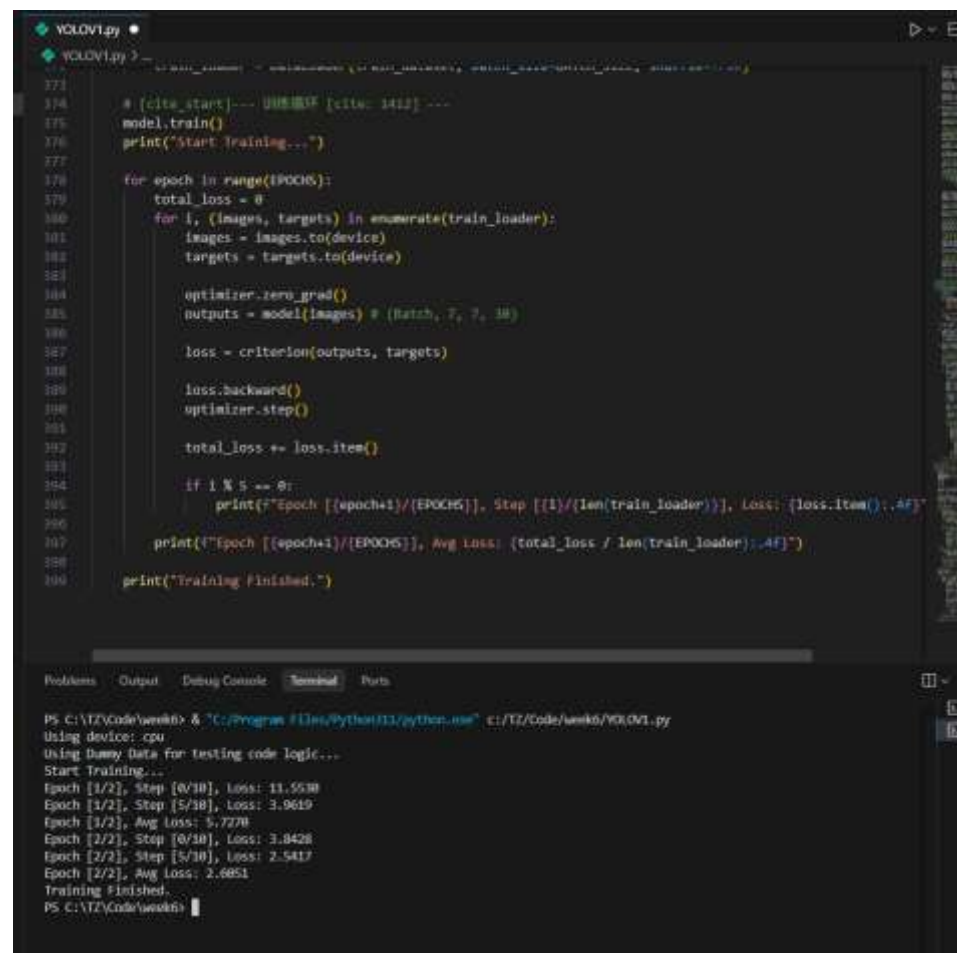
因为每个网格单元只能预测一组类别概率，比如有两个物体（例如一群鸟中的两只鸟）的中心点落在了同一个网格单元内，YOLO v1 无法同时检测出它们。它被迫只能

保留其中一个，另一个会被丢弃。

07.解释“后处理”过程。

“后处理”（Post-processing）是指在 YOLOv1 模型完成前向传播并输出原始数据（ $7 \times 7 \times 30$ 的张量）之后，为了得到最终清晰、准确的检测结果（具体的物体位置、类别和置信度）所进行的一系列计算和筛选过程。简单来说，模型的原始输出包含大量的预测框（98 个），其中很多是重叠的、无效的或低置信度的。后处理的目标就是“去伪存真”。后处理主要用于预测（Inference/Prediction）过程。

08.参考视频内容，将YOLOv1完整代码运行成功并提交运行截图。



```
YOLOv1.py
YOLOv1.py
173 # [cite_start]--- 训练循环 [cite: 3412] ---
174 model.train()
175 print("Start Training...")
176
177 for epoch in range(EPOCHS):
178     total_loss = 0
179     for i, (images, targets) in enumerate(train_loader):
180         images = images.to(device)
181         targets = targets.to(device)
182
183         optimizer.zero_grad()
184         outputs = model(images) # (batch, 7, 7, 30)
185
186         loss = criterion(outputs, targets)
187
188         loss.backward()
189         optimizer.step()
190
191         total_loss += loss.item()
192
193     if i % 5 == 0:
194         print(f"Epoch [{epoch+1}/{EPOCHS}], Step [{i}/{len(train_loader)}], Loss: {loss.item():.4f}")
195
196     print(f"Epoch [{epoch+1}/{EPOCHS}], Avg Loss: {total_loss / len(train_loader):.4f}")
197
198 print("Training Finished.")
```

```
PS C:\TZ\Code\week6> & "C:\Program Files\Python11\python.exe" c:\TZ\Code\week6\YOLOv1.py
Using device: cpu
Using dummy data for testing code logic...
Start Training...
Epoch [1/2], Step [0/10], Loss: 11.5638
Epoch [1/2], Step [5/10], Loss: 3.9619
Epoch [1/2], Avg loss: 5.7278
Epoch [2/2], Step [0/10], Loss: 3.8428
Epoch [2/2], Step [5/10], Loss: 2.5417
Epoch [2/2], Avg loss: 2.6851
Training Finished.
PS C:\TZ\Code\week6>
```